

Multi Factor Authentication

Team number: 5

CS504: Software Engineering

Master of Science in Computer Science

City University of Seattle

Email for All Authors:

- Sumit KC: kcsunit@cityuniversity.edu
- Sai Shruthi Sridhar: sridharsaishruthi@cityuniversity.edu
- Shila Jahanbin: jahanbinshila@cityuniversity.edu

Abstract

Today, keeping online accounts safe is more important than ever. Passwords alone are not sufficient to keep hackers from accessing sensitive data. Multi-Factor Authentication (MFA) increases security by requiring users to provide additional data than simply their password. For example, it may request a fingerprint scan or a code that is sent to their phone. In this project, we studied the workings of MFA and focused on four well-known systems: Duo Security, Okta Verify, Google Authenticator, and Microsoft Azure MFA. We picked these systems because they are widely used in real-world organizations and are known for being user-friendly.

After researching and comparing their features, we found that Duo Security offered the best mix of strong security and easy use. Duo's flexible options, like push notifications, phone calls, and backup codes, make it simple for users to verify their identity without making the login process complicated. This project helped us better understand the important role that user-friendly security solutions play in protecting digital identities today

Keywords: Multi-Factor Authentication (MFA), Cybersecurity, Duo Security, Microsoft Azure MFA, Google Authenticator, Okta Verify, Push Notification Authentication, Mobile Authentication, User-Friendly Security Solutions, Authentication Methods

1. Introduction

In today's digital world, protecting online accounts is more critical than ever. Passwords alone are no longer sufficient to safeguard personal or sensitive data, as attackers are constantly finding new ways to steal or guess credentials. To address this, more robust security measures are needed.

Multi-Factor Authentication (MFA) helps prevent unauthorized access by requiring users to verify their identity using two or more factors such as a password combined with a code sent to their phone or a fingerprint scan.

In this project, we explored the concept of MFA and studied how it improves security in real-world applications. We focused on four widely-used MFA

systems: Microsoft Azure MFA, Google Authenticator, Duo Security, and Okta Verify. Our goal was to compare their strengths and ease of use, and then implement a secure, user-friendly MFA system based on the best practices we identified.

Our solution includes user authentication, One-Time Passcode (OTP) generation and verification, implemented using Flask for the backend and MongoDB for data storage. The frontend is developed using React Native, making the system accessible, flexible, and easy to integrate into small and mid-sized web applications.

2. Literature Review

Multi-Factor Authentication (MFA) has become one of the most effective methods for protecting user identities and sensitive information across

industries. Studies show that relying solely on passwords leaves accounts vulnerable to phishing attacks, credential theft, and brute-force hacking attempts (NIST, 2017). To reduce these risks, MFA adds one or more extra verification **steps**, making it significantly harder for unauthorized users to gain access.

Microsoft Azure MFA is a popular choice for businesses due to its seamless integration with Microsoft 365 and other cloud services (Microsoft, n.d.). It supports authentication methods like mobile notifications and hardware tokens, making it suitable for enterprise-level deployments.

Google Authenticator, often favored by individuals and small teams, generates time-based one-time passwords (TOTP). However, it lacks backup options, which can be a problem if users lose access to their device (Google, n.d.).

Duo Security stands out for its user-friendliness and flexibility. It offers multiple second-factor options, including push notifications, phone calls, and backup codes, making it accessible and secure for various use cases (Duo Security, n.d.).

Okta Verify is well-suited for organizations already using Okta's broader identity management platform. It supports push-based authentication and rotating codes, though its full functionality is best experienced within the Okta ecosystem (Okta, n.d.).

Overall, research indicates that while all of these MFA systems enhance security, factors like ease of use and flexibility heavily influence their adoption. Solutions such as Duo Security, which strike a balance between robust protection and a smooth user experience, are increasingly favored in real-world implementations.

Comparison Table based on the features:

Features	Google Authenticator	Microsoft Azure MFA	Duo Security	Okta Verify
Type	Mobile App	Cloud Based	Cloud Based	Mobile App
Authentication Method	Time-based OTP (TOTP)	Push notification, hardware token	Push notification, calls, backup code	Push notifications, biometrics, rotating codes

Offline Support	With TOTP	With TOTP fallback	With Passcodes	With TOTP
Integration	Manual integration with apps	Microsoft ecosystem and third party apps	VPN, cloud based app, on-premise system	Okta ecosystem and third party apps
Security	Basic two factor authentication	Risk-based checks and blocks suspicious login	Device health and location checks for login	Risk and policy based checks for login
Best use	Personal and two factor authentication	Microsoft ecosystem	Broad MFA support	Okta ecosystem
Cost	Free	Included in Azure Directory Package	Free upto 10 users (Subscription for more)	Included in Okta subscription

Our project uses the lightweight Flask framework for Python to build and manage a Multi-Factor Authentication (MFA) system. The backend includes secure API endpoints for user registration, login, OTP generation, and OTP verification. All user data, including hashed passwords and login timestamps, is stored in MongoDB, which we accessed using the PyMongo library. After evaluating several available MFA solutions, we concluded that Duo Security offers one of the most secure and user-friendly options, especially for real-world applications involving SMS-based authentication.

Educational institutions, entrepreneurs, and developers can all benefit from a reliable Multi-Factor Authentication (MFA) solution. What makes our project stand out is that it's open-source, easy to implement, and freely customizable. This makes it ideal for learning,

experimentation, and real-world use—especially compared to many commercial systems that are often complex or restrictive.

Additionally, our project helps students and early-stage developers bridge the gap between theory and practice by showing how MFA works in an actual application setting.

Our project uses a range of tools for development, management, and collaboration. Flask is a lightweight Python framework that powers the backend and handles routes for user registration, login, OTP generation, and verification.

MongoDB is used as the primary database to store user details, including hashed passwords and login timestamps. We connected to it using the PyMongo driver, which enables seamless interaction between Flask and the database.

Visual Paradigm was used to create UML diagrams, including use case and activity diagrams, which helped visualize the system's structure and user interactions. GitHub served as our version control system, allowing the team to collaborate efficiently and track all changes to the project codebase. We followed the Software Development Life Cycle (SDLC) model throughout the project. This included analyzing requirements, designing the system architecture, implementing core features, testing for correctness and usability, and finally preparing the system for deployment.

(Note: Although Docker is a popular containerization platform, we decided not to use it in this version of the project to keep the setup simple and focused on core MFA functionality.)

3. Methodology

In our project, we built a working Multi-Factor Authentication (MFA) system using Duo Security to protect against common risks like password theft, phishing, and unauthorized access.

The process starts with a simple login using a username and password. Once the user is verified, the system sends a real OTP (one-time passcode) to their phone using Duo's Auth API. This code acts as the second layer of authentication.

To make things smoother, we also connected our system to Duo's Admin API, so whenever a user signs up, they're automatically added to Duo's system with their phone number.

The backend was built using Flask (Python) and the user data, including hashed passwords and login timestamps, is stored in MongoDB. On the

frontend, we used React Native with Expo to create a simple, mobile-friendly interface that walks users through the login and OTP steps.

Using Duo's well-documented APIs gave us a secure, reliable way to manage two-factor authentication, and putting it all together helped us understand how a real-world MFA system actually works behind the scenes.

Figure 1: Activity Diagram

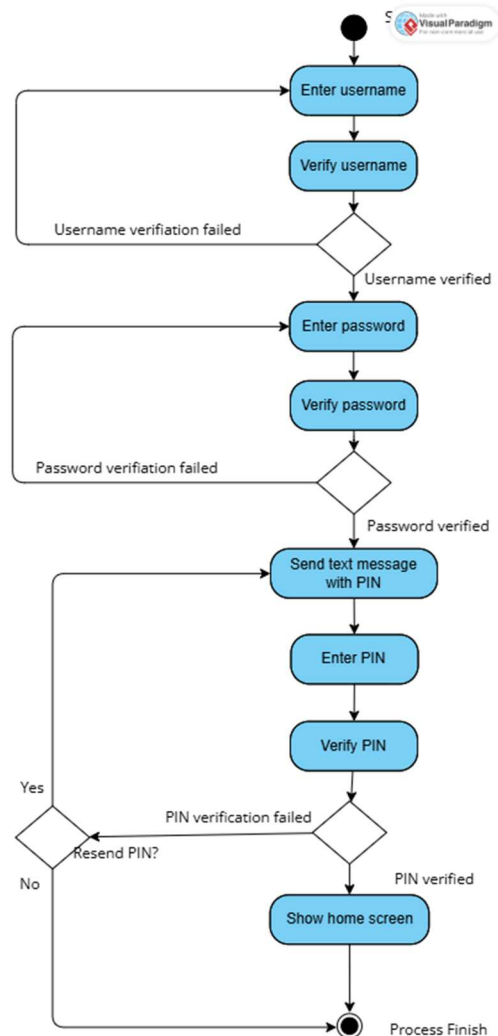


Figure 2: Use Case Diagram

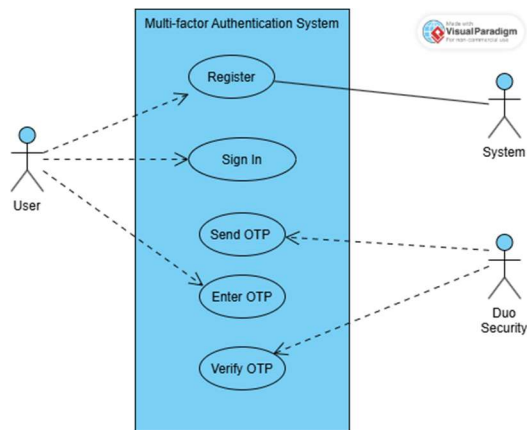


Figure 3: Database Design (NoSQL)

Users	
PK	<u>UserID</u>
	username
	password

Figure 4: Swimlane Diagram

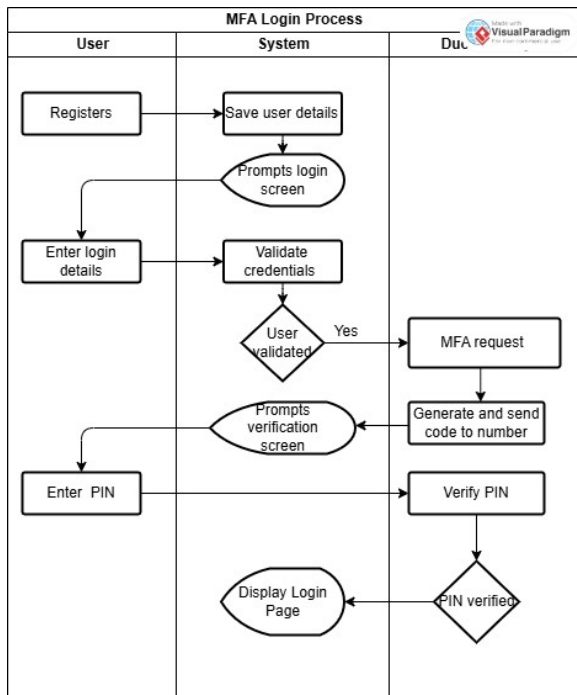
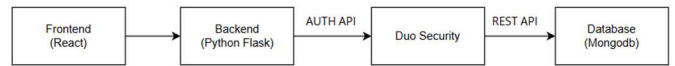


Figure 5: System Architecture



Screenshots of frontend GUI:

Figure 6: Register Page

A screenshot of the Register page. It has a title "Register" and a form with three input fields: "Sai", a blank field, and "+12065790585". Below the fields is a blue "Register" button. At the bottom, it says "Already have an account? [Sign in](#)".

Figure 7: Login Page

A screenshot of the Login page. It has a title "Login" and a subtitle "Welcome to MFA Secure Login". It has two input fields: "Username" and "Password". Below the fields is a blue "Success" message box that says "User registered successfully" with an "OK" button.

Figure 8: OTP sent to phone

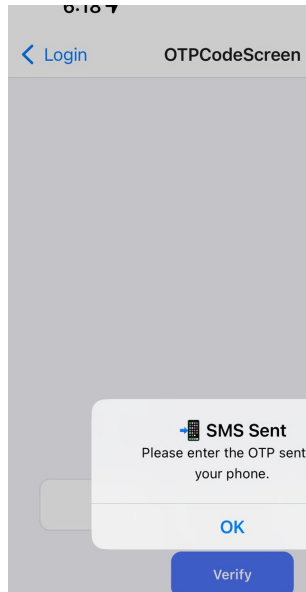


Figure 9: OTP through SMS

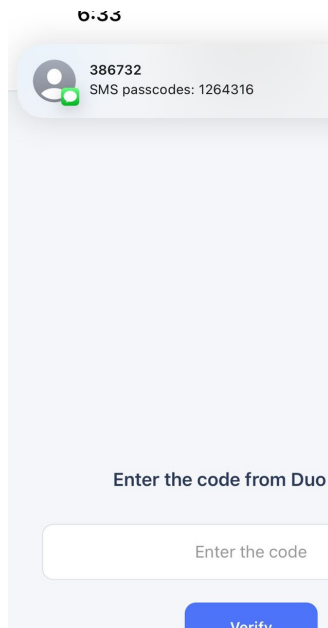
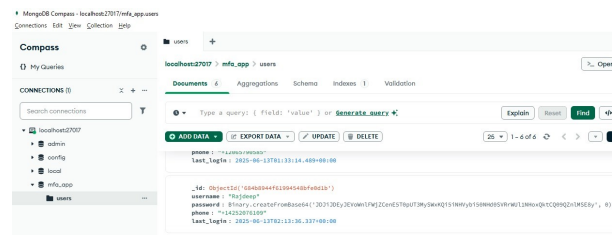


Figure 10: Home Screen



Figure 11: User saved in Database



4. Results

Using the concepts we learned in class, we successfully built a fully working prototype of our Multi-Factor Authentication (MFA) system. The app handles user registration, login, OTP generation, and verification, with complete integration between the Flask backend and React Native frontend.

At first, we tested the OTP process by simulating passcodes in the terminal. Later, we upgraded the system to work with Duo Security's live environment, using their Auth API to send real SMS-based OTP codes to users during login. The Duo integration allowed us to test and validate a real-world MFA flow from start to finish.

The mobile experience was smooth and user-friendly, and we were able to verify that the OTP delivery and verification process worked reliably. In future iterations, we plan to track metrics like login success rate, OTP delivery time, and error rates to evaluate and improve overall performance. These observations will be included in our final report.

5. Conclusion

This project successfully delivers a fully functional Multi-Factor Authentication (MFA) system and highlights the importance of adopting modern authentication techniques in today's digital environment. By exploring, comparing, and finally implementing key elements of widely used MFA solutions, we were able to better understand how authentication systems work not just in theory, but in real-world use cases.

Through hands-on implementation using Flask for backend logic, MongoDB for secure data storage, and Duo Security's Admin and Auth APIs for real-time OTP-based verification, we developed a streamlined and secure login flow. The use of React Native with Expo allowed us to create a cross-platform mobile interface, making the application both accessible and scalable.

In addition to technical development, this project helped us practice collaboration, design thinking, and adherence to the Software Development Life Cycle (SDLC), including requirement analysis, architecture design, implementation, testing, and iteration. These experiences collectively enhanced our understanding of how security systems are built, deployed, and maintained in professional environments.

Moving forward, there is potential to further improve the system by adding features such as push notifications, biometric authentication, or session timeout logic. We also see an opportunity to extend support for more authentication providers, integrate analytics to monitor user login behavior, and eventually dockerize the solution for easier deployment.

Overall, the project not only demonstrates the feasibility of building a secure and flexible MFA system from scratch, but also serves as a practical learning tool for students, developers, and small organizations seeking to adopt strong, real-world cybersecurity practices.

6. Workload Assignments

For this project, the workload was divided equally among all team members to ensure that everyone contributed to the research, writing, and overall success of the assignment.

- **Sumit KC** was responsible for researching Microsoft Azure MFA and drafting the initial comparison between the four authentication systems.
- **Sai Shruthi Sridhar** researched and summarized Google Authenticator and Okta Verify, and helped write the document and also organize the group discussions and meetings.

- **Shila Jahanbin** focused on researching Duo Security, writing the abstract and introduction, and finalizing the literature review section.

We worked together as a team, shared tasks, and helped each other throughout the project to make sure everything came together well.

7. References

- [1] Microsoft. (n.d.). *What is Azure Multi-Factor Authentication?* Microsoft Learn. Retrieved April 27, 2025, from <https://learn.microsoft.com/en-us/azure/active-directory/authentication/concept-mfa-howitworks>
- [2] Google. (n.d.). *Google Authenticator Help*. Google Support. Retrieved April 27, 2025, from <https://support.google.com/accounts/answer/1066447?hl=en>
- [3] Duo Security. (n.d.). *Two-Factor Authentication & Endpoint Security*. Duo Security. Retrieved April 27, 2025, from <https://duo.com/product/trusted-users/two-factor-authentication>
- [4] Okta. (n.d.). *Okta Verify Documentation*. Okta Help Center. Retrieved April 27, 2025, from <https://help.okta.com/en/prod/Content/Topics/end-user/okta-verify-end-user.htm>
- [5] National Institute of Standards and Technology (NIST). (2017). *Digital Identity Guidelines (Special Publication 800-63B)*. Retrieved April 27, 2025, from <https://pages.nist.gov/800-63-3/sp800-63b.html>